

Security challenges in datacenter networks: Risks and Mitigation strategies

Bachelor of Science in Cyber Security and IT Forensics

Final Year Project
Final Report

Stephen Mc Neill Killeen

Dr. Kevin Murphy

23/03/2026

Abstract

Modern datacenter networks support critical cloud and enterprise services making them attractive targets for cyberattacks and network-based threats. Detecting suspicious activity within these environments can be challenging due to the high volume and variety of network traffic generated by different systems.

This project investigates security challenges in datacenter networks and explores mitigation strategies used to protect critical infrastructure. A Python-based log analysis tool was developed to analyse simulated network log data generated from multiple sources including firewall, authentication, intrusion detection system (IDS), and DNS logs. Detection rules were implemented to identify patterns of suspicious activity such as repeated firewall deny events, abnormal authentication attempts, and unusual DNS behaviour. The processed log data was visualised through a monitoring dashboard to provide an overview of network activity and assist with identifying potential security incidents.

The results demonstrate how automated log analysis and monitoring tools can assist security teams in detecting abnormal behaviour and improving visibility within datacenter environments. The project highlights the importance of continuous monitoring and log analysis as part of an effective network security strategy.

Declaration

This report is presented in part fulfilment of the requirements for the Bachelor of Science in Cyber Security and IT Forensics **Final Year Project**.

It is entirely my own work and has not been submitted to any other University or Higher Education Institution or for any other academic award within the University of Limerick.

Where there has been made use of work of other people it has been fully acknowledged and referenced.

Name	Stephen Mc Neill Killeen
Signature	
Date	15/03/2026

Table of Contents

Abstract i

Declaration iii

Table of Contents v

List of Figures vii

List of Tables ix

Chapter 1 – Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Research Aim and Objectives 2

1.4 Research Questions 3

1.5 Scope and Limitations 3

1.6 Ethical Considerations 4

1.7 Structure of the Report 4

Chapter 2 – Motivation for the Project 5

2.1 Importance of Datacenter Security 5

2.2 Challenges in Monitoring Datacenter Networks 6

2.3 Role of Automation in Security Monitoring 7

2.4 Personal Motivation and Project Relevance 7

2.5 Summary of Motivation 8

Chapter 3 – Background Theory and Literature Review 9

3.1 Datacenter Networks 9

3.2 Datacenter Network Architectures 10

3.3 Network Traffic in Datacenters 11

3.4 Virtualisation in Datacenters 12

3.5 Virtual Switches and Internal Networking 13

3.6 Security Monitoring and Log Analysis 14

3.7 Network Security Threats in Datacenters 14

3.8 Mitigation Strategies in Datacenter Security 15

3.9 Security Information and Event Management (SIEM) Systems 16

3.10 Log Analysis Techniques and Approaches 16

3.11 Key Security Technologies and Attack Vectors 17
3.12 Research and Industry Gaps 18
Chapter 4 – System Design and Methodology 21
4.1 Overview of the Project Approach 21
4.2 System Architecture 21
4.3 Log Data Generation 23
4.4 Log Processing and Analysis 24
4.5 Detection Rules 24
4.6 Monitoring Dashboard and UI 25
Chapter 5 – Implementation 27
5.1 System Implementation Overview 27
5.2 Project Structure 27
5.3 Log Generation 28
5.4 Log Parsing and Processing 29
5.5 Detection Rule Implementation 29
5.6 Dashboard Implementation 31
Chapter 6 – Results and Evaluation 33
6.1 Experimental Setup 33
6.2 Detection Results 34
6.3 Dashboard Visualisation 34
6.4 Analysis of Results 36
6.5 Limitations 37
6.6 Summary 38
Chapter 7 – Conclusions and Future Work 39
7.1 Conclusion 39
7.2 Future Work 40
References 41

Appendices 43
Appendix A – Project Gantt Chart 43
Appendix B – Presentation Slides 44
Appendix C – Project Poster 45

List of Figures

Figure 1: Comparison of traditional three-tier and spine-leaf architectures..... 11
Figure 2: North-south and east-west traffic flows..... 12
Figure 3: System architecture of the proposed system..... 22
Figure 4: Internal log processing pipeline..... 24
Figure 5: Monitoring dashboard displaying analysed network activity..... 35

List of Tables

Table 1: Summary of log sources and generated events..... 23

Table 2: Detection rules implemented in the system..... 30

Table 3: Summary of simulated attack scenarios and detection results..... 33

Table 4: Identified system limitations and their impact..... 38

Chapter 1 Introduction

1.1 Background

Datacenters form the backbone of modern digital infrastructure, enabling the operation of cloud computing platforms, enterprise systems, financial services, and large-scale web applications. These facilities consist of highly interconnected networks of servers, storage systems, and networking components that allow organisations to process and manage vast volumes of data in real time. With the rapid growth of technologies such as cloud computing, big data analytics, and Internet of Things (IoT) devices, the demand for reliable and scalable datacenter infrastructure has increased significantly.

Modern datacenters are designed to provide high availability, redundancy, and performance. Technologies such as virtualisation, software-defined networking (SDN), and distributed computing have further increased their complexity. While these advancements improve efficiency and scalability, they also expand the attack surface, making datacenter environments more vulnerable to security threats.

The large-scale and highly dynamic nature of datacenter networks results in continuous flows of network traffic between internal systems and external users. This creates opportunities for a wide range of cyber threats, including Distributed Denial of Service (DDoS) attacks [15], unauthorised access attempts, insider threats, packet sniffing [17], and exploitation of misconfigured services [18]. Attackers often target datacenters due to the high value of the data they host and the critical services they provide.

Security incidents within datacenters can have severe consequences, including service disruption, financial losses, data breaches, and reputational damage. A successful DDoS attack can render services unavailable, while unauthorised access can lead to sensitive data exposure. As a result, ensuring the security of datacenter networks is a critical concern for organisations and service providers.

To mitigate these risks, organisations deploy multiple layers of security controls, including firewalls, intrusion detection and prevention systems (IDS/IPS), encryption protocols, and access control mechanisms. While these systems are effective in enforcing security policies, they generate large volumes of log data that record network events and system activities. Analysing this data is essential for identifying potential threats, but the sheer volume and diversity of logs make manual analysis impractical in large-scale environments.

This challenge has led to increased interest in automated log analysis and monitoring tools, which can process large datasets, identify patterns, and detect anomalies more efficiently than manual approaches.

1.2 Problem Statement

Despite the deployment of multiple security mechanisms, detecting malicious activity within datacenter networks remains a complex and resource-intensive task. Security systems such as firewalls, IDS/IPS, authentication services, and DNS infrastructure continuously generate logs containing detailed records of network events. These logs are often distributed across different platforms and stored in various formats, making it difficult to centralise and analyse them effectively.

Traditional monitoring approaches typically rely on rule-based alerts or manual inspection of logs. While these methods can identify known threats, they often struggle to detect subtle or emerging attack patterns. As datacenter environments grow in scale and complexity, the volume of log data increases significantly, making manual analysis inefficient and prone to oversight.

Many security incidents are only detectable when correlating events across multiple log sources. For example, repeated failed login attempts combined with unusual traffic patterns may indicate a coordinated attack, but such patterns may not be visible when analysing individual logs in isolation.

This project addresses these challenges by exploring the use of lightweight automated log analysis techniques. By processing and analysing log data using a Python-based tool, the project aims to demonstrate how automated monitoring can enhance visibility, improve detection of suspicious activity, and support more effective security management in datacenter environments.

1.3 Research Aim and Objectives

Aim

The aim of this project is to investigate security risks affecting datacenter networks and evaluate mitigation strategies used to protect these environments. Additionally, the project develops a Python-based log analysis and monitoring system to demonstrate how automated analysis of network logs can assist in identifying suspicious activity and improving security visibility.

Objectives

1. Review and classify common network-layer threats affecting datacenter environments.
2. Investigate existing mitigation strategies such as firewalls, IDS/IPS, and network monitoring systems.

3. Develop a Python-based prototype capable of analysing network log data from multiple sources.
4. Implement detection rules to identify patterns of suspicious activity within generated logs.
5. Develop a monitoring dashboard to visualise analysed data and support real-time insights.
6. Evaluate the effectiveness of automated log analysis for improving visibility into datacenter network activity.

1.4 Research Questions

The project aims to address the following research questions:

- What are the primary network-layer security threats affecting modern datacenter environments?
- How effective are existing mitigation strategies such as firewalls and intrusion detection systems in preventing and detecting these threats?
- To what extent can a lightweight Python-based log analysis tool improve the detection of suspicious activity?
- How can automated log analysis contribute to improved visibility and decision-making in datacenter security operations?

1.5 Scope and Limitations

This project focuses on network-layer security within datacenter environments, with particular emphasis on threats such as DDoS attacks, unauthorised access attempts, and abnormal network activity patterns identifiable through system logs. The study examines how these threats can be detected and mitigated using automated log analysis techniques.

However, the scope of the project is limited in several ways. It does not address other aspects of datacenter security, such as physical security controls, organisational policies, or hardware-level vulnerabilities. Additionally, the prototype system developed in this project operates on simulated log data generated in a controlled environment. While this approach allows for structured testing and evaluation, it does not fully replicate the complexity and unpredictability of real-world datacenter environments.

As a result, the findings of this project should be considered as a proof-of-concept demonstration rather than a fully deployable production solution.

1.6 Ethical Considerations

All work conducted in this project adheres to the University of Limerick's ethical research guidelines. No real user data or live network traffic was collected or analysed during the project. Instead, synthetic log data was generated using custom Python scripts within a controlled environment.

This approach ensures that no sensitive information is exposed and that the privacy and security of individuals and organisations are fully protected. The project also avoids any activities that could be considered intrusive or harmful, ensuring compliance with ethical standards in cybersecurity research.

1.7 Structure of the Report

- **Chapter 1** introduces the research topic, outlines the problem, and defines the project aims and objectives.
- **Chapter 2** discusses the motivation and significance of the study in the context of modern cybersecurity challenges.
- **Chapter 3** provides background theory, including datacenter architectures, common network threats, and mitigation strategies.
- **Chapter 4** describes the system design and research methodology, including the architecture of the log analysis tool.
- **Chapter 5** presents the implementation of the Python-based log analysis system and monitoring dashboard.
- **Chapter 6** evaluates the results and discusses their implications for datacenter security monitoring.
- **Chapter 7** concludes the report and outlines recommendations for future work.

Chapter 2 - Motivation for the Project

2.1 Importance of Datacenter Security

Datacenters have become a foundational component of modern digital infrastructure, supporting a wide range of services including cloud computing platforms, enterprise systems, financial transactions, healthcare applications, and government operations. A datacenter is defined as a dedicated facility that houses computing infrastructure such as servers, storage systems, and networking equipment used to process and distribute data [1]. As organisations increasingly migrate services to digital and cloud-based environments, the importance of securing these facilities has grown significantly.

Enterprise datacenters are specifically designed to meet the performance, scalability, and security requirements of large organisations, supporting critical functions such as business continuity, digital transformation, and reliable access to IT systems [2]. However, the centralisation of critical data and services also makes these environments attractive targets for cyber attackers. Attackers may attempt to exploit vulnerabilities to disrupt services, gain unauthorised access, or exfiltrate sensitive data.

The increasing adoption of technologies such as virtualisation, cloud computing, and distributed systems has further expanded the attack surface of datacenter environments. While these technologies improve efficiency and scalability, they introduce additional complexity and make monitoring network activity more difficult. As a result, ensuring the confidentiality, integrity, and availability of datacenter systems has become a major challenge for organisations.

Security breaches in datacenters can have severe consequences, including service outages, financial loss, and reputational damage. Given the critical role datacenters play in supporting global digital services, maintaining strong security controls and monitoring mechanisms is essential. Supporting technologies such as load balancing are commonly used within datacenters to distribute traffic efficiently and maintain performance under high demand [19].

2.2 Challenges in Monitoring Datacenter Networks

One of the primary challenges in securing datacenter environments is maintaining effective visibility into network activity. Modern datacenters generate large volumes of log data from multiple sources, including firewalls, intrusion detection systems, authentication services, and network devices. These logs provide detailed insights into system behaviour and user activity, making them essential for detecting security incidents. However, several challenges complicate the analysis of this data:

Volume and Velocity of Data

Datacenters continuously generate vast amounts of log data due to the scale of operations and the number of interconnected systems. This high volume makes manual analysis impractical and increases the likelihood that important indicators of malicious activity may be overlooked.

Data Distribution and Heterogeneity

Logs are often distributed across multiple systems and stored in different formats. This lack of standardisation makes it difficult to aggregate and analyse data efficiently. Without centralised processing, identifying relationships between events becomes significantly more challenging.

Limited Correlation Between Events

Many cyberattacks involve multiple stages and generate activity across different systems. For example, repeated failed login attempts combined with unusual network traffic may indicate an attack. However, these patterns may not be detected if logs are analysed in isolation.

Limitations of Traditional Monitoring Approaches

Traditional monitoring systems often rely on predefined rules or signatures. While effective for detecting known threats, they may fail to identify new or evolving attack patterns. Additionally, rule-based systems can generate a high number of false positives, placing additional strain on security teams.

Resource Constraints

Manual log analysis requires significant time and expertise. In large-scale environments, security teams may not have the capacity to continuously monitor and analyse all generated data, increasing the risk of delayed threat detection.

2.3 Role of Automation in Security Monitoring

To address the limitations of manual and rule-based approaches, organisations are increasingly adopting automated monitoring and log analysis techniques. Automation enables the rapid processing of large datasets, allowing for the detection of anomalies and suspicious patterns that may not be easily identifiable through manual inspection. Automated systems can perform tasks such as log parsing, pattern recognition, anomaly detection, and alert generation. These capabilities improve the speed and accuracy of threat detection while reducing the reliance on manual processes.

Python has become a widely used language in cybersecurity due to its flexibility and extensive library support. It is commonly used for tasks such as log analysis, network monitoring, and automation of security processes. The ability to quickly develop scripts and tools makes Python particularly suitable for lightweight monitoring solutions. While enterprise-level solutions such as Security Information and Event Management (SIEM) systems provide advanced capabilities, they can be complex and resource-intensive. This creates an opportunity to explore simpler, more accessible alternatives that can still provide meaningful insights into network activity.

2.4 Personal Motivation and Project Relevance

A key motivation for this project is the opportunity to combine theoretical knowledge with practical implementation. During my studies in Cyber Security and IT Forensics, I developed an interest in network security, packet analysis, and the use of programming tools to automate security tasks.

Experience gained during an internship further highlighted the importance of monitoring and analysing network activity in large environments. Observing the challenges associated with

handling large volumes of log data and identifying potential security threats reinforced the relevance of this project.

Python was selected as the implementation language due to its widespread use in cybersecurity and its suitability for rapid development. By developing a lightweight log analysis tool and monitoring dashboard, this project aims to demonstrate how automation can improve visibility and support security monitoring in datacenter environments.

2.5 Summary of Motivation

This project is motivated by the increasing importance of datacenter security and the challenges associated with monitoring complex network environments. The scale, complexity, and dynamic nature of modern datacenters create significant challenges for traditional monitoring approaches.

By investigating these challenges and developing a prototype log analysis system, this project aims to demonstrate how automated techniques can enhance visibility and improve the detection of suspicious activity. The project also provides practical insights into the design and implementation of monitoring tools, bridging the gap between theory and real world application.

Based on the challenges identified, particularly the difficulty of analysing large volumes of distributed log data and the limitations of traditional monitoring approaches, there is a clear need for improved automation in datacenter security. This project addresses these issues by developing a lightweight log analysis system that applies automated processing and detection techniques to network logs. The aim is to demonstrate how such an approach can enhance visibility, improve detection efficiency, and provide practical insights into network behaviour within datacenter environments.

Chapter 3 - Background Theory and Literature Review

3.1 Datacenter Networks

A datacenter is a dedicated facility that houses computing infrastructure such as servers, storage systems, and networking equipment used to support critical digital services. These facilities form the backbone of modern IT systems and are widely used to host cloud platforms, enterprise applications, and large-scale data processing workloads [1], [2].

Datacenters are designed with a strong emphasis on availability, scalability, and resilience. To achieve this, they incorporate redundant power supplies, cooling systems, and fault-tolerant network designs. High availability is particularly important in environments where downtime can lead to significant financial and operational impact. As a result, modern datacenters are engineered to minimise single points of failure and ensure continuous service delivery [4]. In addition, service level agreements (SLAs) often require datacenters to maintain uptime levels exceeding 99.9%, further emphasising the importance of resilient infrastructure design [4].

Enterprise datacenters are specifically tailored to meet the large-scale requirements of organisations. They provide controlled environments where businesses can manage infrastructure, enforce security policies, and ensure compliance with regulatory standards. These datacenters support critical operations such as transaction processing, cloud services, and real-time analytics [2], [4].

However, the increasing scale and complexity of datacenter networks introduce significant security challenges. The large volume of data processed and transmitted within these environments makes them attractive targets for cyber attackers. Additionally, the interconnected nature of systems increases the potential impact of security breaches. As datacenters continue to evolve, maintaining secure and efficient network operations becomes increasingly difficult [1], [4]. The rapid adoption of cloud-based services has increased reliance on datacenter infrastructure, amplifying the consequences of potential disruptions [6].

3.2 Datacenter Network Architectures

The architecture of a datacenter network determines how servers, storage systems, and other devices communicate with each other. Network design has a major influence on performance, scalability, and the ability to monitor network activity. Two commonly discussed architectures in datacenter environments are the traditional three-tier architecture and the modern spine-leaf architecture.

Traditional datacenter networks often follow a three-tier hierarchical design consisting of a core layer, a distribution layer, and an access layer. The access layer connects servers and devices to the network, while the distribution layer aggregates traffic from multiple access switches. The core layer acts as the backbone of the network, forwarding traffic between different parts of the datacenter infrastructure [3], [5]. In this architecture, network traffic typically travels through several layers before reaching its destination, which can introduce additional latency and potential bottlenecks as traffic volumes increase [4]. This design can also lead to oversubscription, where network capacity is insufficient to handle peak traffic loads efficiently [7].

Modern datacenters increasingly use spine-leaf architecture to overcome these limitations. In this topology, leaf switches connect directly to servers and devices, while spine switches provide high-speed connectivity between all leaf switches. Each leaf switch connects to every spine switch, creating multiple equal-cost paths for traffic and improving network scalability and reliability [6]. This design reduces latency and provides more predictable performance, particularly in environments with large amounts of server-to-server communication [6], [7]. It also supports horizontal scaling, allowing additional capacity to be added without major architectural changes.

However, the distributed nature of spine-leaf architectures can also create challenges for security monitoring. Because traffic is spread across multiple network paths and devices, security logs and events may be generated in many different locations, making it more difficult to gain a complete view of network activity [7]. This increases the importance of centralised log collection and analysis systems.

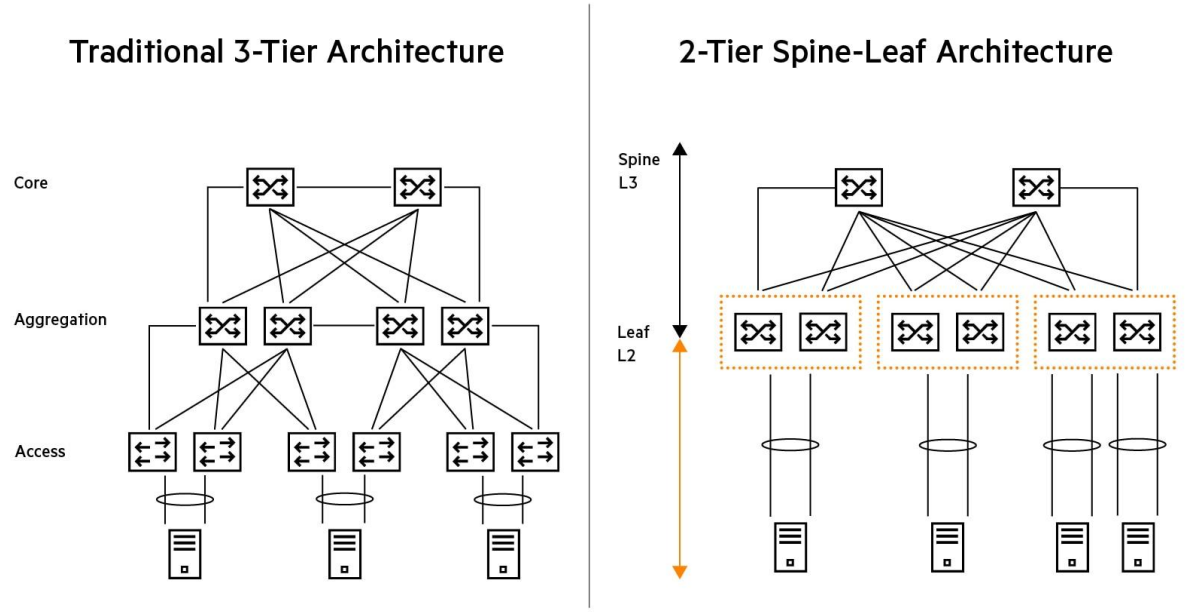


Figure 1: Comparison of traditional three-tier and spine-leaf datacenter architectures [6]

3.3 Network Traffic in Datacenters

Network traffic in datacenter environments can generally be classified into two main categories: north-south traffic and east-west traffic.

North-south traffic refers to communication between external systems and internal datacenter resources. This type of traffic typically passes through perimeter security devices such as firewalls, load balancers, and intrusion detection systems. Because this traffic crosses the network boundary, it is often easier to monitor and inspect using traditional security controls [4]. These controls allow organisations to enforce policies such as access restrictions and traffic filtering at defined entry points.

East-west traffic refers to communication that occurs internally within the datacenter, such as server-to-server or virtual machine-to-virtual machine communication. As modern applications become more distributed, east-west traffic has increased significantly [4]. However, this type of communication may bypass perimeter security devices, making it more

difficult to detect malicious activity using traditional monitoring methods. Attackers who gain access to a system can exploit this internal traffic to move laterally across the network [4].

As a result, analysing logs from internal systems becomes an important method for identifying suspicious behaviour within datacenter networks [4]. Effective monitoring of east-west traffic is therefore critical for detecting insider threats and advanced persistent attacks.

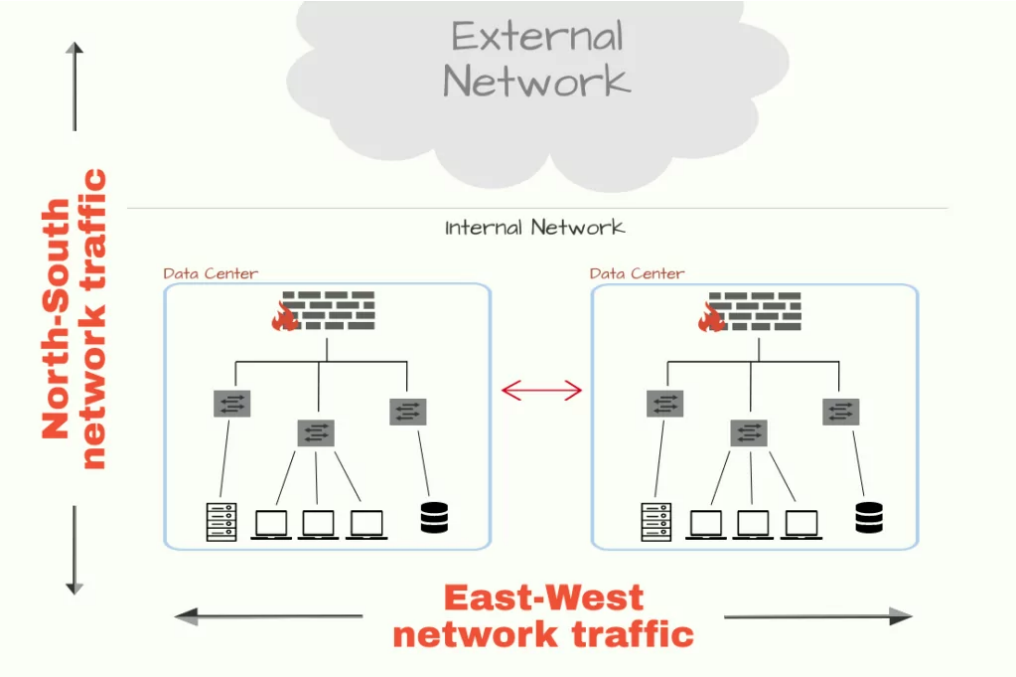


Figure 2: Illustration of north–south and east–west traffic flows in a datacenter [20]

3.4 Virtualisation in Datacenters

Virtualisation is widely used in modern datacenter environments to improve resource utilisation and scalability. Server virtualisation allows a single physical server to host multiple virtual machines (VMs), each running its own operating system and applications. This approach enables organisations to deploy services more efficiently while reducing hardware costs and improving flexibility [8].

In virtualised environments, a software layer known as a hypervisor manages the allocation of physical hardware resources such as CPU, memory, storage, and network interfaces. The hypervisor ensures that each virtual machine operates independently while sharing the

underlying hardware resources [8]. This abstraction layer is essential for cloud computing environments and dynamic workload management.

Virtualisation significantly increases the amount of network activity within datacenters because multiple virtual machines may communicate internally within the same physical host. This can generate large volumes of network logs and increase the complexity of monitoring network behaviour [8]. Additionally, the dynamic nature of virtual environments, where virtual machines can be rapidly created and removed, introduces further challenges for tracking system activity.

3.5 Virtual Switches and Internal Networking

In virtualised datacenter environments, communication between virtual machines is often handled by virtual switches (vSwitches). A virtual switch is a software-based network switch that performs similar functions to a physical network switch, including packet forwarding, traffic inspection, and network policy enforcement [8].

Virtual switches allow virtual machines to communicate with each other and with external networks through physical network interfaces. Because some communication between virtual machines occurs entirely within the hypervisor environment, this traffic may not pass through traditional network monitoring devices [8].

As a result, security events occurring within virtualised infrastructure may only appear in virtual switch logs or hypervisor logs. This distributed logging environment increases the complexity of monitoring network activity and highlights the importance of analysing logs from multiple sources when investigating potential security incidents [8]. Without visibility into virtual traffic, attackers may exploit these environments to remain undetected.

3.6 Security Monitoring and Log Analysis

Security monitoring is an essential component of protecting datacenter networks from cyber threats. Security tools such as firewalls, intrusion detection systems, and authentication services generate logs describing network events, connection attempts, and potential security alerts [13], [14].

In large datacenter environments, these logs can quickly accumulate into extremely large volumes of data. Because logs are generated by many different systems and devices, correlating events across multiple log sources becomes increasingly difficult. Manual analysis of these logs is often impractical, particularly in environments with high network traffic and large numbers of devices [13]. This challenge is further compounded by the variety of log formats and the lack of standardisation across systems.

Automated log analysis tools provide a potential solution to this challenge by identifying patterns of suspicious behaviour within log data. By analysing indicators such as repeated connection attempts, unusual traffic spikes, or abnormal authentication activity, automated tools can assist security teams in detecting potential threats more quickly and improving overall visibility into network activity [14].

The need for automated log analysis forms the basis for the practical component of this project, where a lightweight Python-based tool is developed to analyse network log data and identify suspicious activity patterns. Such tools provide a flexible and accessible alternative to large-scale enterprise monitoring systems.

3.7 Network Security Threats in Datacenters

Datacenter environments are exposed to a wide range of network security threats due to their scale, complexity, and the critical services they support. One of the most significant threats is Distributed Denial of Service (DDoS) attacks, which aim to overwhelm network resources and disrupt service availability. These attacks can generate extremely high volumes of traffic, often exceeding the capacity of network infrastructure and causing service outages [15].

Another common threat is brute-force and unauthorised access attempts. In these attacks, malicious actors repeatedly attempt to gain access to systems using guessed or stolen credentials. These activities can often be identified through patterns in authentication logs, such as repeated failed login attempts from the same source [16].

Packet sniffing and traffic interception also pose risks within datacenter networks. Attackers may capture network traffic to extract sensitive information, particularly if encryption is not properly implemented. This type of attack highlights the importance of secure communication protocols and monitoring tools capable of identifying suspicious traffic patterns [17].

In addition, misconfigurations in network devices and security controls are a significant source of vulnerabilities. Incorrect firewall rules, weak access controls, or exposed services can provide attackers with entry points into the network. Such issues are often difficult to detect without proper monitoring and auditing mechanisms [18]. These threats demonstrate the need for continuous monitoring and analysis of network activity, particularly through log data, to identify suspicious behaviour and respond to potential security incidents.

3.8 Mitigation Strategies in Datacenter Security

To defend against network threats, organisations implement multiple layers of security controls designed to protect datacenter infrastructure. Firewalls are commonly used to filter network traffic and enforce access control policies. They act as a barrier between trusted internal networks and untrusted external sources, blocking unauthorised connections [9], [10].

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) monitor network activity for suspicious behaviour. IDS systems generate alerts when anomalies are detected, while IPS systems can actively block malicious traffic [12].

Encryption protocols such as Transport Layer Security (TLS) are used to protect data in transit, ensuring confidentiality and integrity. Access control mechanisms, including authentication and authorisation systems, further restrict access to authorised users only.

However, these traditional security measures have limitations. Many attacks occur within the network and may not be detected by perimeter-based controls. Additionally, rule-based detection systems may struggle to identify new or evolving threats. This highlights the need for more advanced monitoring approaches that incorporate behavioural analysis and anomaly detection [13], [14].

3.9 Security Information and Event Management (SIEM) Systems

Security Information and Event Management (SIEM) systems are widely used in enterprise environments to collect, analyse, and correlate log data from multiple sources. These systems provide a centralised platform for monitoring network activity and detecting potential security incidents [13], [14].

SIEM systems aggregate logs from various devices, including firewalls, servers, and applications, allowing security teams to identify patterns and relationships between events. They also provide features such as real-time alerting, reporting, and compliance monitoring. Despite their advantages, SIEM systems have several limitations. They can be complex to configure, require significant computational resources, and often generate a high number of false positives. Additionally, the cost of deployment and maintenance can be a barrier for smaller organisations [13].

These limitations highlight the need for alternative approaches, such as lightweight log analysis tools, which can provide valuable insights without the complexity of full-scale enterprise systems.

3.10 Log Analysis Techniques and Approaches

Log analysis is a critical process in cybersecurity, involving the examination of system and network logs to identify patterns, anomalies, and indicators of compromise. Several approaches are commonly used in log analysis.

Rule-based detection involves defining specific conditions that indicate potential threats, such as multiple failed login attempts or unusual traffic volumes. While effective for detecting

known threats, this approach may not identify new or unknown attack patterns [14]. Pattern recognition techniques focus on identifying recurring behaviours within log data. For example, repeated access attempts from a single IP address may indicate a brute-force attack [16].

Anomaly detection involves identifying deviations from normal behaviour. This approach can be effective in detecting previously unknown threats, but it may also generate false positives if normal behaviour is not well defined [14]. Log correlation is another important technique, where events from multiple sources are analysed together to identify relationships between them. This is particularly useful in datacenter environments, where attacks often involve multiple systems and stages [13]. These techniques form the basis for automated log analysis systems, including the Python-based tool developed in this project.

3.11 Key Security Technologies and Attack Vectors in Datacenters

Datacenter security relies on a combination of technologies that work together to protect network infrastructure and detect malicious activity. Understanding these technologies and the types of attacks they are designed to prevent is essential for effective monitoring and analysis.

Firewalls

Firewalls are one of the most fundamental security mechanisms used in datacenter networks. They control incoming and outgoing traffic based on predefined security rules, acting as a barrier between trusted and untrusted networks [9], [10],[11]. Firewalls are particularly effective at preventing unauthorised access and filtering malicious traffic. However, they are typically deployed at network boundaries and may not detect threats that occur within the internal network.

Intrusion Detection and Prevention Systems (IDS/IPS)

Intrusion Detection Systems (IDS) monitor network traffic for suspicious activity and generate alerts when potential threats are identified. Intrusion Prevention Systems (IPS) extend this functionality by actively blocking malicious traffic [12]. These systems are effective for

detecting known attack patterns but may struggle with new or evolving threats that do not match predefined signatures.

Authentication and Access Control

Authentication systems are used to verify the identity of users and devices attempting to access network resources. Common methods include password-based authentication, multi-factor authentication (MFA), and token-based systems [16]. Strong authentication mechanisms are essential for preventing unauthorised access. However, weak credentials or poor configuration can lead to vulnerabilities such as brute-force attacks.

Domain Name System (DNS) Security

The Domain Name System (DNS) is responsible for translating domain names into IP addresses. While essential for network communication, DNS can also be exploited by attackers. Common DNS-related threats include DNS spoofing, cache poisoning, and tunnelling, where attackers use DNS traffic to bypass security controls [10]. Monitoring DNS logs can provide valuable insights into suspicious behaviour, such as connections to malicious domains.

3.12 Research and Industry Gaps

Despite significant advancements in datacenter security technologies, several limitations remain evident in both academic research and industry practice. Traditional security mechanisms such as firewalls and intrusion detection systems (IDS/IPS) continue to rely heavily on predefined rules and signature-based detection methods. While these approaches are effective for identifying known threats, they often struggle to detect new or evolving attack patterns, particularly in dynamic and large-scale environments [16], [17].

In enterprise environments, Security Information and Event Management (SIEM) systems are commonly used to aggregate and analyse log data from multiple sources. Although SIEM platforms provide powerful capabilities for log correlation and real-time monitoring, they are often complex to configure, resource-intensive, and can generate a high volume of false positives [25]. This can make them difficult to manage effectively, particularly for smaller organisations or environments with limited resources.

Another key challenge is the lack of lightweight and accessible monitoring solutions. Much of the existing research focuses on large-scale enterprise systems or advanced detection techniques, such as machine learning-based approaches, which may not always be practical to implement in smaller environments. There is comparatively less focus on simple, flexible tools that can provide meaningful insights without requiring significant infrastructure or cost.

These gaps highlight the need for alternative approaches that balance effectiveness with simplicity. This project addresses this gap by developing a lightweight Python-based log analysis tool that demonstrates how automated monitoring can be implemented in a practical and accessible way. By focusing on multi-source log analysis and rule-based detection, the project aims to provide a simplified model of how datacenter monitoring systems can improve visibility and support threat detection.

Chapter 4 - System Design and Methodology

4.1 Overview of the Project Approach

This project combines academic research with a practical prototype implementation to explore security monitoring challenges within datacenter networks. The research component investigates common network threats and mitigation strategies, while the practical component demonstrates how automated log analysis can assist in detecting suspicious activity within complex network environments.

As discussed in earlier chapters, datacenter infrastructures generate large volumes of distributed log data from multiple sources, including firewalls, authentication systems, intrusion detection systems (IDS), and DNS infrastructure. These logs contain valuable information regarding network activity, user behaviour, and potential security incidents. However, due to their distributed nature and scale, manual analysis of such logs is inefficient and difficult to maintain in large environments [13]. As discussed in Chapter 3, the ability to correlate events across multiple log sources is essential for detecting coordinated or multi-stage attacks [14].

To address these challenges, this project adopts a pipeline based approach to log analysis. A Python-based system is designed to ingest, process, and analyse log data from multiple sources, applying rule-based detection techniques to identify suspicious patterns. The results are then presented through a web based monitoring dashboard, providing an accessible interface for analysing detected events.

This approach reflects real-world datacenter monitoring systems, where automation is used to improve visibility, reduce manual workload, and enable faster detection of potential threats.

4.2 System Architecture

The architecture of the proposed system is designed to simulate a simplified datacenter monitoring environment in which multiple sources generate log data that is centrally

analysed. The system is structured as a series of interconnected components that collectively perform ingestion, processing, detection, and visualisation.

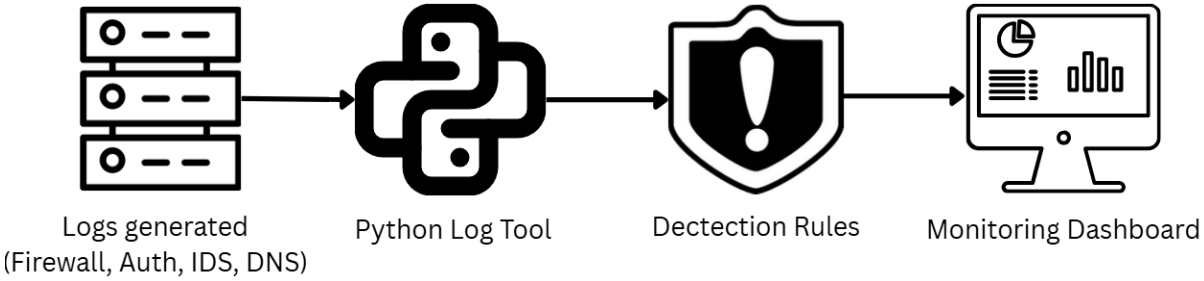


Figure 3: System architecture of the proposed log analysis and monitoring system

The architecture begins with multiple simulated log sources, representing key components of a datacenter environment, including firewall systems, authentication services, intrusion detection systems, and DNS infrastructure. These sources produce log files containing records of network events and user activity.

The log data is then processed by a central analysis system implemented in Python. During this stage, the system reads the input data, applies field mappings, and converts logs from different formats into a unified structure. This normalisation process is essential, as it allows logs from heterogeneous sources to be analysed consistently.

Following normalisation, the processed data is passed to a detection layer where rule-based algorithms are applied. These rules are designed to identify patterns that may indicate malicious behaviour, such as repeated access attempts or abnormal traffic activity. The results of this analysis are then stored as structured outputs, including flagged events and summary statistics.

The final component of the architecture is the presentation layer, which consists of a web-based dashboard implemented using Flask. This interface allows users to view and interact with the analysed data, providing visual insights into network activity and detected anomalies.

This layered architecture reflects the distributed and multi-source nature of datacenter monitoring, where effective analysis requires the aggregation and processing of logs from multiple independent systems [14].

4.3 Log Data Generation

To evaluate the effectiveness of the proposed system, simulated log data was generated using custom Python scripts. This approach allows for controlled experimentation while avoiding the ethical and security risks associated with analysing real network traffic.

The generated logs represent four key sources commonly found in datacenter environments: firewall logs, DNS logs, IDS alerts, and authentication logs. Each source captures a different aspect of network activity. Firewall logs record connection attempts and access control decisions, DNS logs capture domain resolution requests, IDS logs represent detected security alerts, and authentication logs record user access attempts.

In addition to normal activity, the system includes simulated attack scenarios designed to replicate realistic threat behaviour. These scenarios include patterns resembling brute-force login attempts, traffic spikes indicative of denial-of-service activity, and unusual DNS query behaviour. By incorporating such patterns into the generated data, the system can be evaluated against known indicators of suspicious activity.

The use of synthetic data provides several advantages. It enables repeatable testing, allows specific scenarios to be examined in isolation, and ensures compliance with ethical guidelines by avoiding the use of sensitive or real-world data.

Table 1: Summary of log sources and generated events

Log Source	Description	Example Events
Firewall	Monitors network traffic	Allow/Deny connections
DNS	Tracks domain queries	Suspicious domains
IDS	Detects attacks	Alerts triggered
Authentication	User login activity	Failed logins

4.4 Log Processing and Analysis

Once log data has been generated, it is processed by the analysis system. This stage involves parsing raw log entries, extracting relevant attributes, and transforming the data into a structured format suitable for analysis.

A key aspect of this process is the normalisation of log data. Because each log source uses different formats and field names, the system applies mappings to convert all logs into a common schema. This unified structure includes fields such as timestamps, source and destination addresses, event types, and other relevant attributes. By standardising the data, the system is able to analyse logs from different sources using a consistent methodology.

In addition to normalisation, the system performs aggregation and classification of events. For example, network traffic is categorised based on its direction, allowing the system to distinguish between internal and external communication. This is particularly important in datacenter environments, where internal (east–west) traffic can be more difficult to monitor than external (north–south) traffic [4].

The automated processing of log data significantly improves efficiency compared to manual analysis. In large-scale environments, where logs are generated continuously by numerous systems, automated tools are essential for identifying patterns and anomalies within large datasets [13], [14].

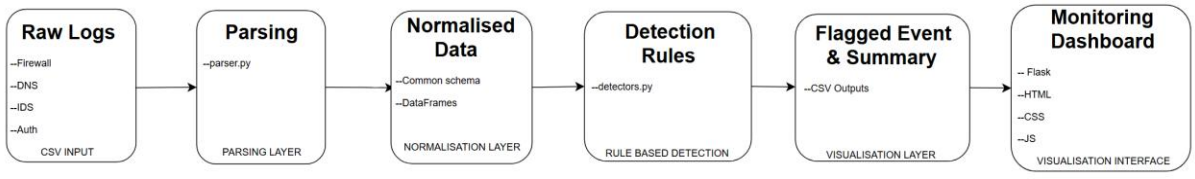


Figure 4: Internal log processing pipeline showing data ingestion, parsing, normalisation, detection, and visualisation stages

4.5 Detection Rules

Following processing, the system applies rule-based detection techniques to identify suspicious behaviour within the log data. These rules are designed to detect patterns that deviate from expected activity and may indicate potential security incidents.

The detection methodology is based on analysing temporal and behavioural patterns within the data. For example, repeated failed authentication attempts within a short time period may indicate a brute-force attack, while a sudden increase in network traffic may suggest a

denial-of-service event. Similarly, unusual DNS query patterns may indicate command-and-control communication or data exfiltration attempts.

The system applies detection across multiple log sources, allowing it to identify patterns that span different parts of the network. This multi-source analysis improves visibility and increases the likelihood of detecting coordinated attacks. As discussed in Chapter 3, the ability to correlate events across systems is a key requirement for effective security monitoring [16].

The detection thresholds used by the system are configurable, allowing the analysis to be adjusted based on different scenarios or environments. This flexibility reflects real-world monitoring systems, where detection rules must be tuned to balance sensitivity and false positives [17].

4.6 Monitoring Dashboard and UI

To present the results of the analysis, a web-based monitoring dashboard was developed using the Flask framework. The dashboard serves as the interface between the user and the analysis system, allowing results to be viewed and interpreted more effectively.

The front-end of the dashboard is implemented using HTML templates, CSS styling, and JavaScript for dynamic content rendering. HTML is used to structure the layout of the interface, while CSS is applied to improve visual clarity and usability. JavaScript enables interaction with the backend, allowing data to be loaded and updated dynamically without requiring full page reloads.

The dashboard provides multiple views corresponding to different log sources, enabling users to examine firewall activity, authentication events, IDS alerts, and DNS queries individually. It also presents summaries of detected events, allowing users to quickly identify patterns of interest.

The integration of a visual interface significantly enhances the usability of the system. Rather than analysing raw log files, users can interact with structured and visualised data, making it easier to identify trends and anomalies. Visualisation is widely recognised as an important component of security monitoring systems, as it improves situational awareness and supports faster decision-making.

Chapter 5 - Implementation

5.1 System Implementation Overview

The practical component of this project involved the development of a Python-based log analysis system designed to simulate security monitoring within a datacenter environment. The system processes network log data generated from multiple sources and applied rule-based detection techniques to identify patterns of suspicious activity.

Python was selected as the primary implementation language due to its flexibility and extensive support for data processing and automation tasks. It is widely used in cybersecurity applications for tasks such as log analysis, scripting, and automation, making it suitable for developing lightweight monitoring tools [14]. The system was developed as a modular application, allowing different components such as log generation, processing, detection, and visualisation to operate independently.

The implementation focuses on demonstrating how automated log analysis can improve visibility into network activity and assist in identifying potential security threats within complex environments. The system integrates multiple stages of processing, reflecting real-world monitoring pipelines used in datacenter security systems.

5.2 Project Structure

The project was organised into several directories, each representing a key component of the system. This modular structure improves maintainability and allows individual components to be extended or modified without affecting the entire system.

The main directories within the project include:

- src/
- generators/
- Dashboard/
- data/
- outputs/

The src directory contains the core log analysis engine, including scripts responsible for parsing log data, processing events, and applying detection rules. Key files include main.py, which acts as the central execution point of the system, parser.py, which handles log normalisation, and detectors.py, which contains the detection logic.

The generators directory contains scripts used to create simulated network log data representing different types of activity, including both normal behaviour and attack scenarios. The Dashboard directory contains the web-based interface, implemented using Flask, along with HTML templates, JavaScript, and CSS files used for visualisation.

The data directory stores input log files, while the outputs directory contains processed results, including flagged events and summary data. This separation of input and output data improves clarity and reflects common practices in data processing systems.

5.3 Log Generation

To simulate network activity within a datacenter environment, custom Python scripts were developed to generate synthetic log data. These scripts create realistic log entries representing both normal network behaviour and potential attack scenarios.

Several generator scripts were implemented, including basic test data generation, DDoS-like traffic simulation, external ransomware activity simulation, and insider data exfiltration scenarios. These generators produce logs across multiple sources, including firewall, DNS, IDS, and authentication systems.

The generated logs are structured to reflect real-world formats, including timestamps, IP addresses, ports, and event types. This allows the analysis system to process the data in a way that closely resembles real monitoring environments.

The inclusion of attack simulation scenarios is particularly important for evaluating the effectiveness of the detection system. By generating patterns such as traffic spikes or repeated login attempts, the system can be tested against known indicators of malicious

behaviour. This aligns with common practices in cybersecurity research, where synthetic datasets are used to evaluate detection systems in controlled environments [13].

5.4 Log Parsing and Processing

Once log data is generated, it is processed by the Python log analysis engine. The system reads log files and extracts relevant information from each entry, including timestamps, event types, and network-related attributes.

The parsing process converts raw log data into structured formats that can be analysed efficiently. This functionality is implemented within the parser.py module, where field mappings are applied to standardise logs from different sources into a unified schema. This schema includes common attributes such as source IP, destination IP, timestamps, and event indicators.

The system also utilises a pipeline-based structure, where logs from different sources are processed through dedicated pipelines. Each pipeline is tailored to the specific characteristics of the log source, ensuring accurate parsing and processing. For example, authentication logs include user-related fields, while DNS logs focus on query and response data.

In addition to parsing, the system performs classification of traffic, distinguishing between internal and external communication. This enables the identification of east–west traffic, which is often more difficult to monitor but critical for detecting lateral movement within networks [10].

Automated log processing enables the system to analyse large volumes of data efficiently and is a key requirement for effective security monitoring in datacenter environments [13], [14].

5.5 Detection Rule Implementation

After processing, the system applies rule-based detection techniques to identify suspicious activity within the log data. These rules are implemented within the detectors.py module and operate by analysing patterns across multiple log entries.

The detection rules are designed to identify behaviours that may indicate potential security threats, including repeated firewall deny events, unusual DNS activity, multiple failed authentication attempts, and IDS alerts indicating possible intrusion.

An example of a detection rule implemented in the system is shown below:

```
#Detect repeated firewall denies
flagged_denies = detect_repeated_denies(
   df_firewall,
   window_minutes=w_brute,
   threshold=fw_threshold_denies
)
```

This rule identifies repeated connection attempts within a defined time window, which may indicate brute-force attacks or automated scanning activity. Similar functions are implemented for detecting authentication anomalies, DNS patterns, and IDS alert spikes.

The detection system is configurable through external configuration files, allowing thresholds and parameters to be adjusted without modifying the core code. This design improves flexibility and reflects real-world systems where detection rules must be tuned based on network behaviour.

Rule-based detection provides a simple but effective method for identifying known patterns of malicious behaviour within network logs. Although more advanced techniques such as machine learning exist, rule-based systems remain widely used due to their interpretability and ease of implementation [17].

Table 2: Detection rules implemented in the system

Rule	Description	Example Threat
Repeated Denies	Multiple blocked attempts	scanning
Failed Logins	Repeated auth failures	Brute force
DNS Anomaly	Unusual queries	C2 communication
IDS Alerts	Triggered signatures	Intrusion

5.6 Dashboard Implementation

To present the results of the analysis, a web-based monitoring dashboard was developed using the Flask framework. The dashboard provides a visual interface that allows users to explore network activity and review detected security events.

The dashboard reads output data generated by the analysis system and displays it through multiple pages, each corresponding to a specific log source. These include firewall, DNS, IDS, and authentication views.

The dashboard is implemented using a combination of backend and frontend technologies. The backend, written in Python using Flask, handles routing, data processing, and communication with the analysis system. It exposes endpoints that allow the frontend to retrieve processed data and trigger analysis tasks.

The frontend is built using HTML templates, CSS for styling, and JavaScript for dynamic interaction. HTML templates define the structure of each page, CSS improves visual presentation, and JavaScript enables dynamic updates, allowing data to be displayed without reloading the page. This combination creates an interactive interface that enhances usability. The dashboard also includes functionality for loading generated data, displaying summaries, and presenting detected anomalies in a structured format. This allows users to quickly identify patterns and investigate potential security incidents.

The use of visualisation significantly improves the effectiveness of the system. Rather than analysing raw log files, users can interact with processed data, making it easier to detect trends and anomalies. This approach reflects modern security monitoring systems, where dashboards are used to provide real-time insights into network activity [13].

Chapter 6 – Results and Evaluation

6.1 Experimental Setup

To evaluate the effectiveness of the developed log analysis system, simulated network log data was generated using the custom Python log generators described in Chapter 5. These generators produced log data representing both normal network behaviour and potential attack scenarios within a datacenter environment.

The simulated logs included four primary sources: firewall logs, DNS logs, intrusion detection system (IDS) alerts, and authentication logs. Each of these sources represents a key component of datacenter monitoring, providing visibility into different aspects of network activity.

Different scenarios were generated to simulate suspicious behaviour, including repeated connection attempts, abnormal login activity, and unusual DNS patterns. These scenarios were designed to reflect common network-layer threats such as brute-force attacks, scanning activity, and anomalous network communication [15], [16]. In addition, more complex scenarios such as traffic spikes and insider-style behaviour were generated to evaluate how the system responds to varying patterns of activity.

The generated log data was then processed using the Python analysis tool, where detection rules were applied to identify patterns of suspicious activity. The use of synthetic datasets allowed for controlled testing of specific behaviours, ensuring that detection mechanisms could be evaluated against known patterns while maintaining ethical compliance.

Table 3: Summary of simulated attack scenarios and detection results

Scenario	Description	Logs Affected
DDoS	High Traffic spikes	Firewall
Brute Force	Login Attempts	Auth
DNS Abuse	Suspicious queries	DNS
Intrusion	IDS alerts	IDS

6.2 Detection Results

The log analysis system successfully identified multiple patterns of suspicious behaviour within the generated log data. Detection rules implemented within the system were able to flag events across different log sources, demonstrating the effectiveness of the rule-based approach.

The system identified repeated firewall deny events occurring within short time intervals, which are commonly associated with scanning activity or unauthorised access attempts. Similarly, multiple failed authentication attempts were detected, indicating possible brute-force attacks. DNS logs also revealed unusual query patterns, which may suggest abnormal communication behaviour or potential command-and-control activity. IDS alerts generated by the system highlighted potential intrusion attempts, further supporting the effectiveness of multi-source monitoring.

The system's ability to process log data from multiple sources and detect patterns across them is a key strength. Many attacks involve multiple stages and may not be identifiable when analysing logs in isolation. By correlating events across firewall, authentication, DNS, and IDS logs, the system provides improved visibility into network behaviour, which is essential for detecting coordinated or distributed attacks [14].

In addition, the detection system was able to identify temporal patterns, such as bursts of activity within defined time windows. This demonstrates the effectiveness of time-based detection thresholds implemented within the system.

6.3 Dashboard Visualisation

The results of the log analysis were presented through the monitoring dashboard, which provides a visual representation of network activity and detected events. The dashboard allows users to view summaries of flagged events and analyse trends across different log sources.

The dashboard includes multiple views, each corresponding to a specific log type, as well as a main overview page displaying aggregated information. Visual elements such as graphs and charts are used to represent network activity over time and highlight anomalies. These visualisations are generated from processed output data and dynamically displayed using the dashboard interface.

The integration of the Flask backend with HTML templates, CSS styling, and JavaScript allows the dashboard to present data in an interactive format. Users can navigate between different log views and observe patterns across multiple datasets, improving situational awareness.

The use of visualisation improves the usability of the system by allowing users to quickly identify patterns and investigate suspicious behaviour without needing to manually analyse raw log data. This reflects common practices in modern monitoring systems, where dashboards are used to provide real-time insights into network activity [13].

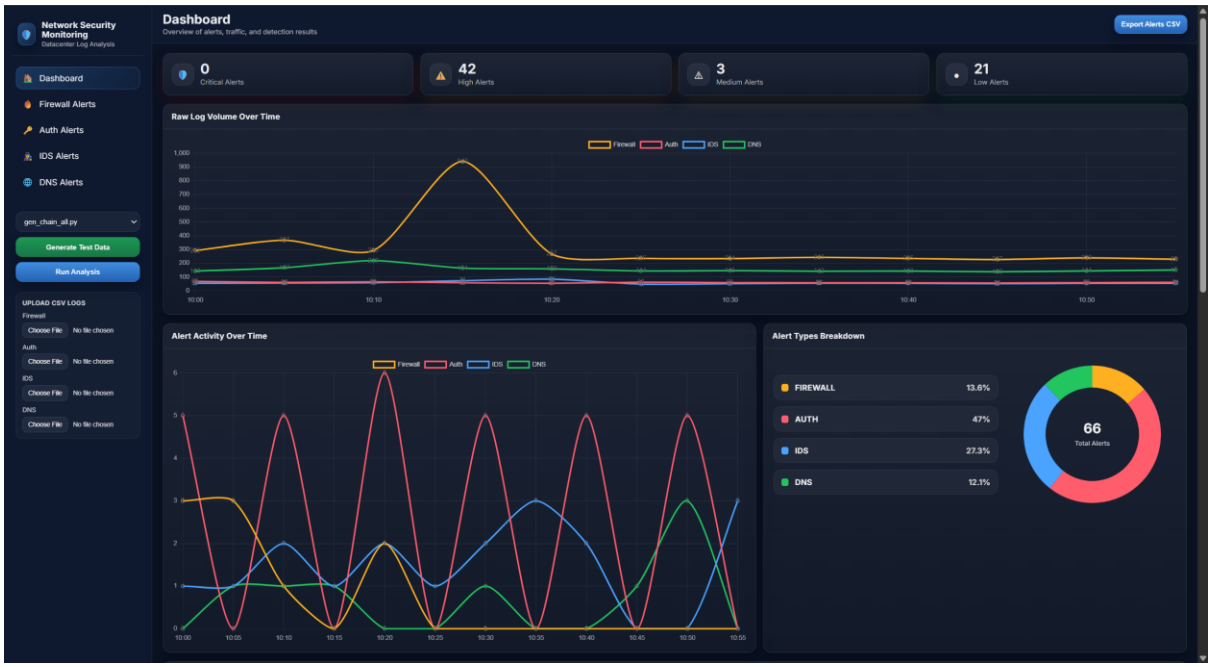


Figure 5: Monitoring dashboard displaying analysed network activity and detected anomalies.

6.4 Analysis of Results

The results demonstrate that automated log analysis can effectively identify patterns of suspicious activity within datacenter environments. By processing and analysing log data from multiple sources, the system provides improved visibility into network behaviour.

The detection rules implemented in the system were effective in identifying known patterns of malicious activity, such as repeated connection attempts and abnormal login behaviour. These patterns are commonly associated with network attacks and can serve as early indicators of potential security incidents [16]. The ability to detect such patterns across multiple log sources reinforces the importance of log correlation in security monitoring.

A key strength of the system is its modular design, which allows different log sources to be analysed independently while still contributing to a unified detection process. This reflects real-world monitoring environments, where data is collected from diverse systems and must be integrated to provide meaningful insights.

Furthermore, the system demonstrates that relatively simple rule-based techniques can still provide valuable detection capabilities when applied effectively. While advanced detection methods such as machine learning are increasingly used, rule-based systems remain widely adopted due to their transparency and ease of implementation [14].

The inclusion of traffic classification, such as distinguishing internal and external communication, also enhances the system's analytical capability. This is particularly important for identifying lateral movement within networks, which is a common characteristic of advanced attacks [4].

The findings of this evaluation directly address the research questions outlined in Chapter 1. The system successfully demonstrates that common network-layer threats, such as brute-force attacks, scanning activity, and abnormal DNS behaviour, can be identified through log analysis. This supports the understanding of key threats affecting datacenter environments.
In terms of mitigation strategies, the results highlight the importance of monitoring systems that complement traditional security controls such as firewalls and IDS/IPS. While these

controls provide essential protection, the analysis shows that additional visibility can be achieved through log correlation and automated detection.

Furthermore, the implementation confirms that a lightweight Python-based log analysis tool can effectively improve the detection of suspicious activity within a controlled environment. The system demonstrates that even simple rule-based techniques can provide meaningful insights when applied across multiple log sources. Overall, the results support the conclusion that automated log analysis can enhance decision-making and improve situational awareness in datacenter security monitoring.

6.5 Limitations

While the system demonstrates the potential of automated log analysis, several limitations should be noted.

Firstly, the system operates on simulated log data rather than real-world network traffic. Although this allows for controlled testing, it does not fully capture the complexity, noise, and unpredictability of real datacenter environments. In practice, real logs may contain incomplete data, inconsistencies, or unexpected behaviours that are difficult to replicate in synthetic datasets.

Secondly, the detection approach is based on predefined rules. While effective for identifying known patterns of suspicious behaviour, rule-based systems may struggle to detect previously unknown or highly sophisticated attacks. Advanced threats may evade detection if they do not match predefined patterns, highlighting the limitations of purely rule-based approaches [14].

Another limitation relates to scalability. The system is designed as a lightweight prototype and processes log data from local files. In real-world environments, log data is generated continuously at large scale, requiring distributed processing systems and real-time analysis capabilities.

Finally, while the dashboard provides useful visualisation, it is limited in terms of advanced analytics and real-time updates. Enterprise monitoring systems often include more sophisticated features such as alert prioritisation, automated response mechanisms, and integration with other security tools. These limitations reflect some of the risks identified during the initial project planning phase, particularly those related to scalability, data realism, and the constraints of implementing a prototype system within a limited timeframe.

Table 4: Identified system limitations and their impact.

Limitation	Impact
Simulated Data	Not fully realistic
Rule-based Detection	Misses unknown attacks
Limited Scale	Not suitable for enterprise-scale deployment

6.6 Summary

This chapter has demonstrated that the developed Python-based log analysis system is capable of processing network log data and identifying patterns of suspicious activity. The use of detection rules and visualisation tools provides an effective method for improving visibility into network behaviour within datacenter environments.

The evaluation highlights both the strengths and limitations of the system. While the approach is effective for detecting known patterns and demonstrating the benefits of automated monitoring, further enhancements would be required for deployment in real-world environments.

Overall, the results support the conclusion that automated log analysis can play a valuable role in improving datacenter security monitoring and assisting in the detection of potential security threats.

Chapter 7 - Conclusions and future work

7.1 Conclusion

This project investigated the security challenges associated with modern datacenter networks and explored mitigation strategies used to protect critical infrastructure. As datacenters continue to support cloud computing and enterprise services, maintaining visibility into network activity remains a significant challenge due to the volume, velocity, and complexity of generated data.

To address this challenge, a Python-based log analysis system was developed to simulate automated monitoring within a datacenter environment. The system processed log data from multiple sources, including firewall, DNS, IDS, and authentication logs, and applied rule-based detection techniques to identify patterns of suspicious activity. The implementation reflects real-world monitoring scenarios, where multiple independent systems generate logs that must be analysed collectively to detect potential threats.

The results demonstrated that the system was capable of detecting behaviours such as repeated connection attempts, abnormal authentication activity, and unusual DNS patterns. These findings directly address the research questions outlined in Chapter 1, showing that network-layer threats can be effectively identified through log analysis and that automated tools can improve detection capabilities compared to manual approaches.

The implementation of a monitoring dashboard further enhanced the system by providing a visual interface for analysing network activity. This highlights the importance of combining automated detection with visualisation tools to improve usability and support security analysis. The use of a web-based interface also reflects common practices in modern security operations, where dashboards are used to provide real-time insights into network behaviour.

A key contribution of this project is the demonstration that lightweight, Python-based tools can provide meaningful insights into datacenter security without the complexity of large-scale enterprise systems. While advanced platforms such as SIEM solutions offer extensive capabilities, this project shows that simpler approaches can still be effective in identifying suspicious patterns and improving visibility.

Overall, the project demonstrates that automated log analysis can improve visibility into datacenter network activity and assist in the identification of potential security threats. While the system is a simplified prototype, it provides a practical and technically grounded demonstration of how monitoring tools can support network security in complex environments.

7.2 Future Work

Although the developed system successfully demonstrates the use of automated log analysis, several improvements could be made to enhance its capabilities and extend its applicability to real-world environments.

One potential improvement is the integration of real-time log processing. The current system operates on pre-generated log files, whereas real-world monitoring systems require continuous analysis of live data streams. Implementing real-time data ingestion, for example through streaming technologies or log forwarding mechanisms, would allow the system to detect threats as they occur and improve its responsiveness.

Another area for improvement is the use of advanced detection techniques such as machine learning or anomaly detection. While rule-based detection is effective for identifying known patterns, more advanced methods could enhance the system's ability to detect previously unseen or evolving threats. For example, anomaly detection models could be used to establish baseline behaviour and identify deviations that may indicate malicious activity.

Further enhancements could include expanding the range of log sources analysed by the system. Modern datacenter environments generate logs from additional components such as application servers, cloud platforms, and endpoint devices. Incorporating these sources would provide a more comprehensive view of network activity and improve detection accuracy.

Scalability is another important consideration for future development. The current implementation processes log data locally, which is suitable for demonstration purposes but may not be sufficient for large-scale environments. Future work could explore the use of distributed processing frameworks or cloud-based architectures to handle higher volumes of data.

Integration with existing security platforms, such as Security Information and Event Management (SIEM) systems, could also improve the practical applicability of the solution. This would allow the system to complement existing monitoring tools and contribute to a broader security infrastructure.

Finally, future work could focus on enhancing the dashboard interface by incorporating more advanced visualisations and interactive features. This could include real-time updates, alert prioritisation, and more detailed filtering options to support deeper analysis of network activity.

References

[1] Wikipedia Contributors, “Data center,” *Wikipedia*, Sep. 17, 2019. https://en.wikipedia.org/wiki/Data_center

[2] “The Digital Realty Blog,” *Digitalrealty.ie*, 2026. https://www.digitalrealty.ie/resources/articles/an-in-depth-guide-to-enterprise-data-centres

[3] “3-Tier Architecture Data Center or Traditional Data Center :: Services :: Kaigen Group,” *www.kaigengroup.com*. https://www.kaigengroup.com/three-tier-datacenter/

[4] J. Diamond, “Data Center Network Architecture - Key Components, Challenges and Insights,” *Park Place Technologies*, Oct. 28, 2024. https://www.parkplacetechnologies.com/blog/data-center-network-architecture/

[5] “Comparing Two-Tier and Three-Tier Data Center Networks,” *www.wwt.com*. https://www.wwt.com/article/comparing-two-tier-three-tier-data-center-networks

[6] “What is spine-leaf architecture? | Glossary,” *Hpe.com*, 2026. https://www.hpe.com/ie/en/what-is/spine-leaf-architecture.html

[7] S. Stoklosa, “Understanding the Difference: Spine-Leaf vs. Traditional Data Center Architectures - STORDIS,” *STORDIS*, Nov. 07, 2024. https://stordis.com/spine-leaf-vs-traditional-data-center-architectures/

[8] vmware, “What is a Hypervisor? | VMware Glossary,” *www.vmware.com*, 2024. https://www.vmware.com/topics/hypervisor

[9] “What is a Data Center Firewall? | VMware Glossary,” *Vmware.com*, 2024. https://www.vmware.com/topics/data-center-firewall

[10] “What Is a Perimeter Firewall?,” *Palo Alto Networks*. https://www.paloaltonetworks.com/cyberpedia/what-is-a-perimeter-firewall

[11] “What Is a Distributed Firewall?,” *Palo Alto Networks*. https://www.paloaltonetworks.com/cyberpedia/what-is-a-distributed-firewall

[12] *Trellix.com*, 2025.
https://www.trellix.com/security-awareness/network/ids-vs-ips-key-differences-explained/

[13] K. Kent and M. Souppaya, "Special Publication 800-92 Guide to Computer Security Log Management Recommendations of the National Institute of Standards and Technology," NIST, Sep. 2006. Available:
https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf

[14] SANS Institute, "Log Analysis Techniques."
https://www.sans.org/white-papers/log-analysis

[15] Cloudflare, "What is a DDoS attack?," *Cloudflare*, 2025.
https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/

[16] OWASP, "Brute Force Attack Software Attack | OWASP Foundation," *owasp.org*.
https://owasp.org/www-community/attacks/Brute_force_attack

[17] "Packet Sniffing - an overview | ScienceDirect Topics," *Sciencedirect.com*, 2014.
https://www.sciencedirect.com/topics/computer-science/packet-sniffing

[18] "NVD - Home," *nvd.nist.gov*.
https://nvd.nist.gov

[19]IBM, "Load balancing," *Ibm.com*, Oct. 11, 2021.
https://www.ibm.com/think/topics/load-balancing

[20] "Why Is It Important? East-West and North-South Traffic Security," *SOCRadar® Cyber Intelligence Inc.*, Sep. 10, 2020.
https://socradar.io/blog/east-west-and-north-south-traffic-why-is-it-security-important/

Appendices

Appendix A – Project Gantt Charts ..... Appendices - 3

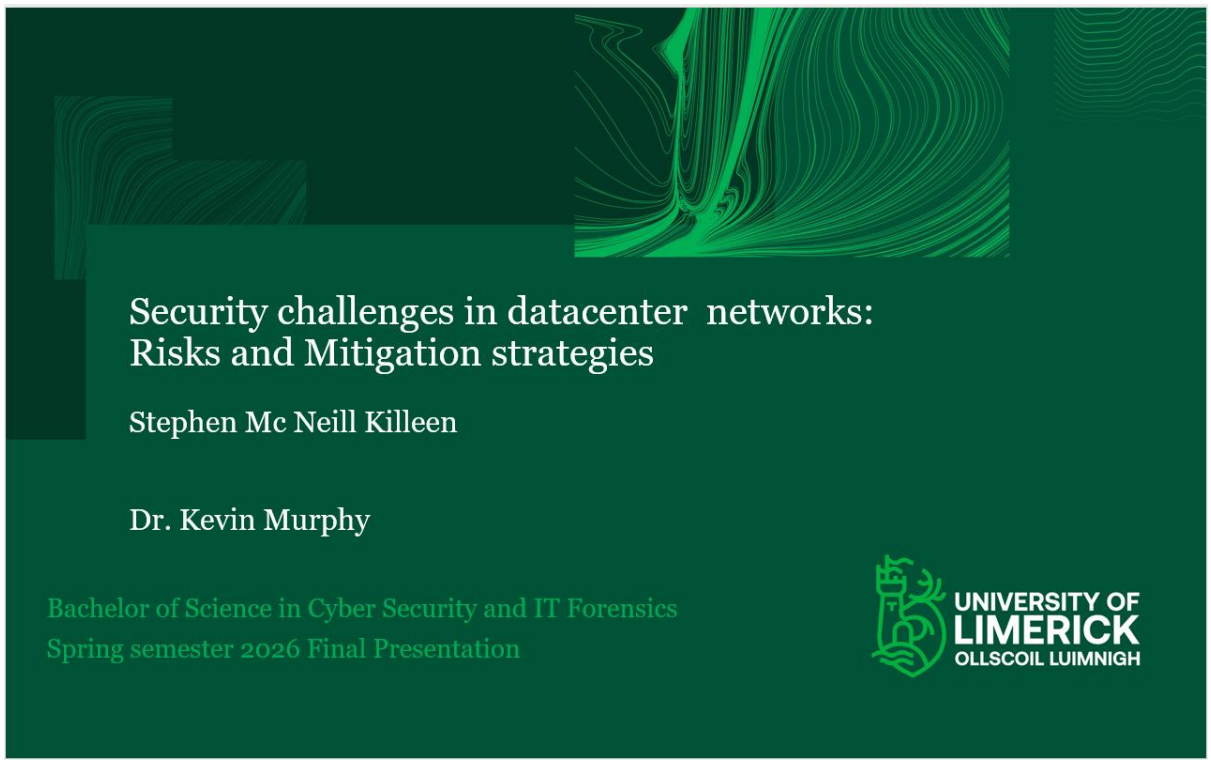
Appendix B – Presentation Slides ..... Appendices - 5

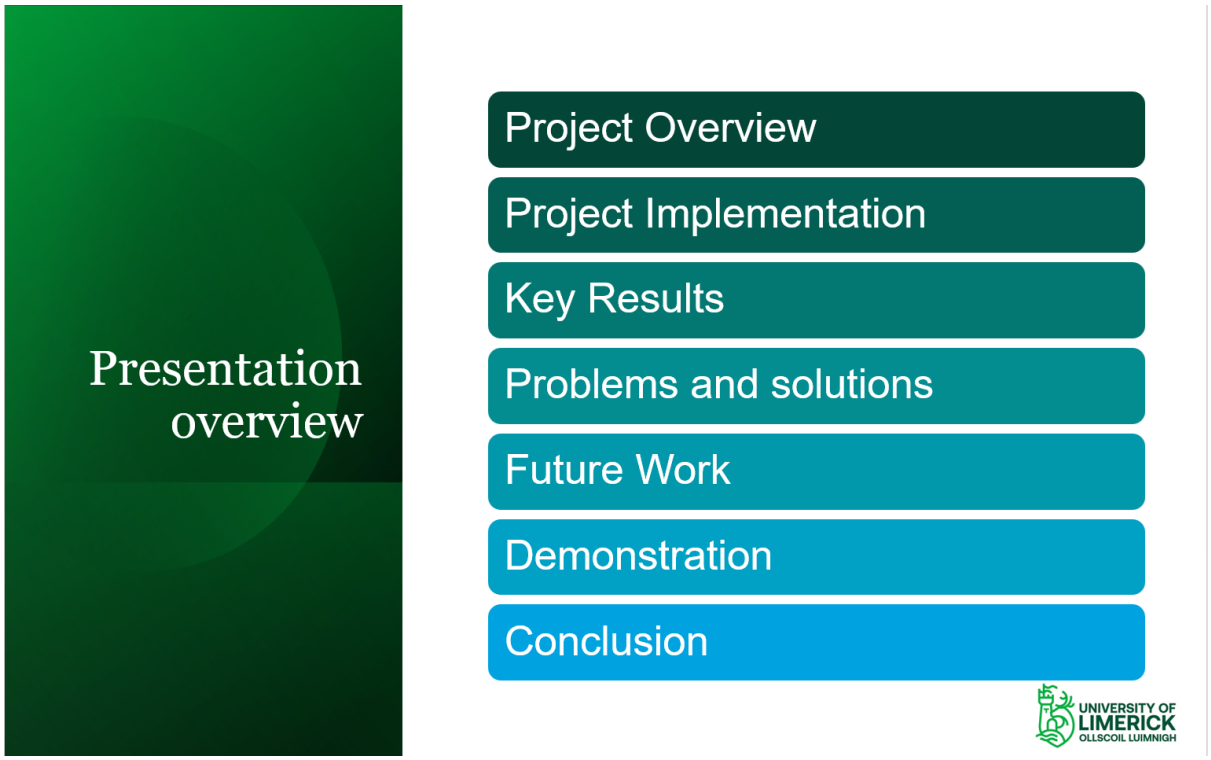
Appendix C – Project poster ..... Appendices – 13

Appendix A: Updated project Gantt chart

Week	Dates	Task	Description
Week 1	26-Jan	Review & Planning	Reviewed interim feedback, refined project scope and system design
Week 2	02-Feb	System Design	Designed architecture for log analysis tool and dashboard
Week 3	09-Feb	Initial Development	Began implementation of log generators and basic scripts
Week 4	16-Feb	Core Development	Developed parsing system and structured log processing pipeline
Week 5	23-Feb	Detection Implementation	Implemented rule-based detection (firewall, auth, DNS, IDS)
Week 6	02-Mar	System Integration	Integrated modules (generators, parser, detectors, outputs)
Week 7	09-Mar	Testing & Debugging	Tested system with simulated attacks and refined detection rules
Week 8	16-Mar	Evaluation & Report Writing	Analysed results and complete both the final report and presentation slides.
Week 9	23-Mar	Finalisation & Submission	Report submitted and prepare for the presentation on Wednesday 25th.

Appendix B: Final presentation slides







Project overview – Introduction

- Modern datacenter networks generate large volumes of distributed network traffic, making security monitoring increasingly complex
- Traditional monitoring approaches struggle to analyse large-scale, multi-source log data efficiently
- This project develops a Python-based log analysis system to simulate automated security monitoring
- The system processes firewall, DNS, IDS, and authentication logs to detect suspicious behaviour
- The aim is to demonstrate how automated log analysis improves visibility and threat detection

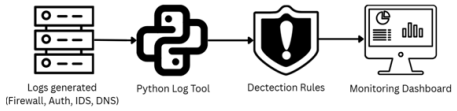
UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Project overview - Functionality

- Simulates a datacenter monitoring environment
- Aggregates logs from multiple distributed sources
- Applies rule-based detection techniques
- Identifies suspicious patterns across network activity
- Visualises results using a monitoring dashboard

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Project implementation - Architecture



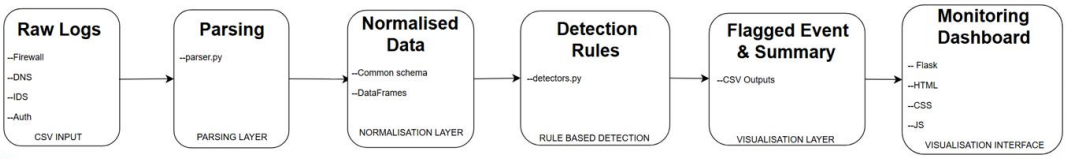
System Architecture

- The system is designed as a modular log analysis architecture
- Multiple log sources are simulated, including firewall, DNS, IDS, and authentication systems
- Logs are collected and processed centrally by the Python analysis engine
- This approach allows correlation of events across different network components
- The analysed results are then visualised through a monitoring dashboard

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Project implementation – Log Processing

- Raw log data is generated and stored in text-based formats
- A parsing layer extracts key attributes such as timestamps, IP addresses, and event types
- Logs are converted into structured formats (e.g. DataFrames) for efficient analysis
- Data is normalised into a common schema to handle different log formats
- This enables consistent processing across all log sources



UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Log Processing Pipeline

Project implementation - Detection

- Detection is performed using rule-based analysis of log data
- The system identifies patterns that deviate from expected behaviour
- Examples of detection rules include:
  - Repeated firewall deny events (potential scanning)
  - Multiple failed login attempts (brute-force attacks)
  - Unusual DNS query patterns
  - IDS alerts indicating possible intrusion
- Detection uses configurable thresholds and time windows to improve accuracy

Example detection rule implementation

```
#Detect repeated firewall denies
flagged_denies = detect_repeated_denies(
    df_firewall,
    window_minutes=w_brute,
    threshold=fw_threshold_denies
)
```

Key results

- The system successfully detected multiple types of suspicious behaviour
- Repeated firewall deny events indicated potential scanning activity
- Multiple failed login attempts identified possible brute-force attacks
- Unusual DNS activity highlighted abnormal network behaviour
- IDS alerts were captured and correlated with other log sources
- Combining multiple log sources improved detection accuracy and visibility

Key results



Monitoring Dashboard Output

- Real-time alert overview
- Visualisation of log activity trends
- Breakdown of alert types
- Supports quick identification of anomalies

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Problems and solutions

PROBLEMS

- High volume of distributed log data
- Inconsistent log formats
- Manual analysis inefficiency
- Risk of false positives

SOLUTIONS

- Automated log processing pipeline
- Parsing and data normalisation
- Rule-based detection engine
- Threshold tuning and filtering

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Potential uses of the system and future work

Uses:

- Lightweight log monitoring solution
- Supports analysis of network activity
- Demonstrates SIEM like functionality
- Can integrate with enterprise systems

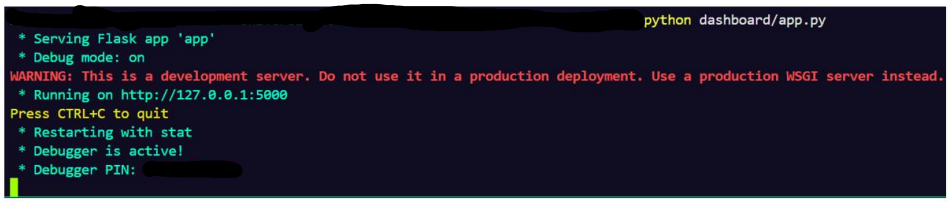
Future Work:

- Real-time log processing
- Machine learning / anomaly detection
- Additional log sources (cloud, endpoints)
- Improved dashboard visualisation

UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Demonstration

- Generate simulated log data (firewall, DNS, IDS, authentication)
- Execute the Python log analysis system
- Apply detection rules to identify suspicious behaviour
- Display flagged events and summaries in the monitoring dashboard

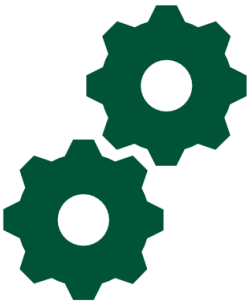


System execution output

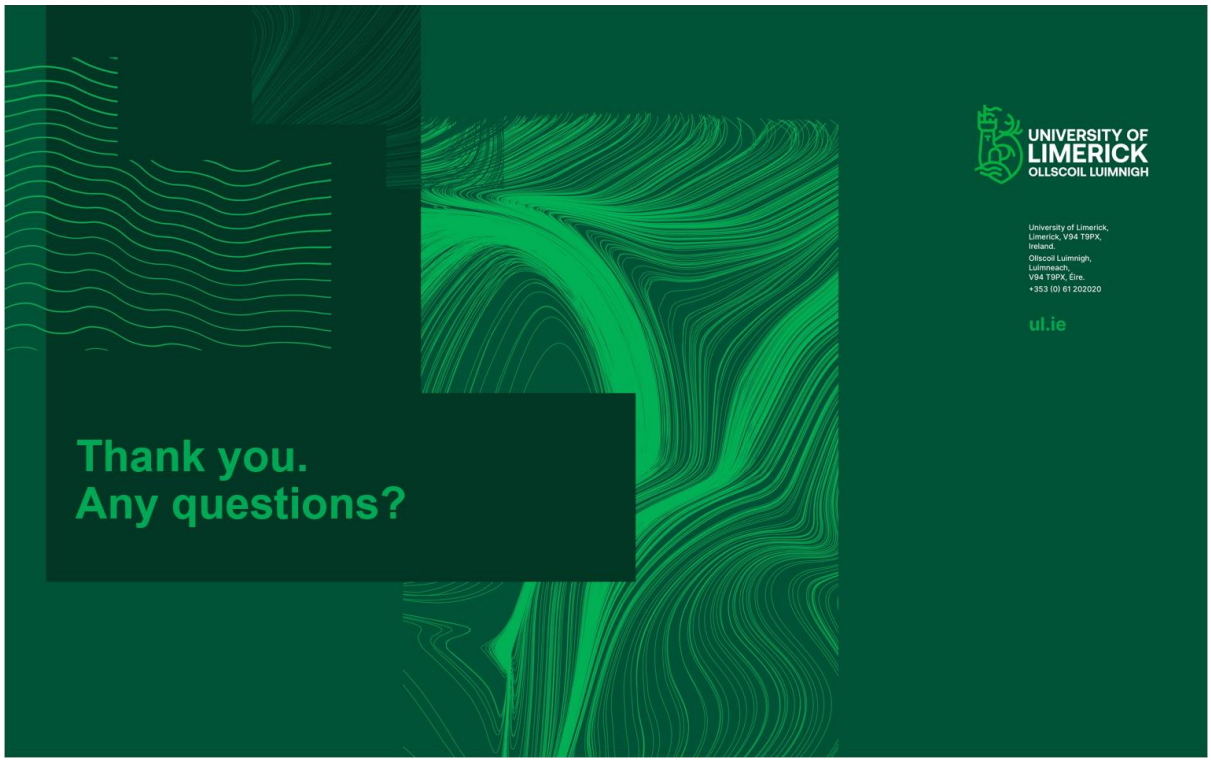
UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

Conclusions

- Datacenter environments produce large volumes of distributed log data
- Automated log analysis improves visibility and reduces manual effort
- Rule-based detection effectively identifies common attack patterns
- The developed system demonstrates a practical approach to network monitoring
- Future improvements can enhance scalability and detection capabilities



UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH



Appendix C: Project poster

Security challenges in datacenter networks: risks and mitigation strategies

UNIVERSITY OF LIMERICK OLLSCOIL LUIMNIGH | Department of Electronic and Computer Engineering

Stephen Mc Neill Killeen

Cyber Security & IT Forensics

Introduction

Modern datacenter networks support cloud computing, enterprise services and critical online infrastructure. Due to their scale and complexity, they are frequent targets for cyberattacks which include Distributed Denial of Service (DDoS), unauthorised access, ransomware and malware activity.

This project investigates key security challenges in datacenter networks and explores mitigation strategies used to protect them. For This project I have developed a Python-based log analysis tool and monitoring dashboard to demonstrate how automated monitoring can assist in identifying suspicious activity within a datacenter network.

Aim

The aim of this project is to research the security challenges affecting modern datacenter networks and examine mitigation strategies used to protect critical infrastructure and services. As datacenters support large-scale cloud and enterprise systems, they are frequent targets for cyber threats and it is crucial to show how effective monitoring and log analysis is a vital part of protecting this infrastructure.

Method

System Implementation

The python-based log analysis system was developed to simulate security monitoring within a datacenter environment. Custom Python scripts generate simulated network logs representing activity from multiple sources including firewalls, authentication systems, DNS servers, and intrusion detection systems (IDS).

The generated logs are processed and normalised using the Python analysis tool, where predefined detection rules analyse log entries to identify suspicious activity patterns such as repeated firewall denies, abnormal login attempts, or unusual DNS behaviour.

Log Analysis Process

The developed tool analyses simulated network log data to identify potentially suspicious behaviour within datacenter environments. Log entries are processed using a python-based analysis tool where predefined detection rules are applied to identify abnormal patterns such as repeated login attempts, unusual DNS activity, or firewall alerts.

Detected events are then visualised through a monitoring dashboard, allowing network administrators to quickly identify and investigate potential security incidents.

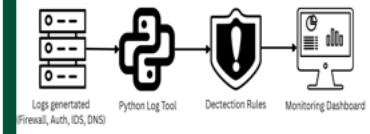


Figure 1: Architecture of the log analysis system

Detection Workflow

- Network logs are generated for firewalls, authentication systems, DNS servers, and IDS alerts through a custom log generator made in python
- The logs are then processed and then normalised using the Python log analysis tool
- Detection rules are then applied to identify suspicious patterns such as repeated firewall denies or login failures
- Flagged events are then stored stored and passed to the monitoring dashboard for visualisation

```
--- repeated denies ---
flagged_denies = detect_repeated_denies(
    df_firewall,
    window_minutes=w_brute,
    threshold=fw_threshold_denies
)
```

Figure 2: Detection rule used to identify repeated firewall deny events within network logs

Results

The developed Python log analysis tool successfully processed simulated network log data through log generators I made in python for firewall, Auth, IDS and DNS logs.

It Identified patterns of suspicious activity through the use of detection rules that were applied to analyse events such as repeated firewall deny actions, abnormal authentication attempts and unusual DNS activity.

The monitoring dashboard displays analysed log events and highlights potentially malicious behaviour. This allows administrators to quickly identify patterns of unusual activity and investigate potential security incidents within the network environment. There is a main dashboard and individual pages for each log source, where each page has the functionality of reviewing both the raw log data and the data in a normalised format.



Figure 3: Monitoring Dashboard

Event Analysis

Analysis of the generated network log data identified multiple patterns of suspicious activity across different log sources. The detection rules successfully flagged repeated firewall deny events, abnormal authentication attempts, and unusual DNS activity.

Visualising these events allows security analysts to identify trends in network behaviour and quickly detect anomalies. The results demonstrate how automated log analysis can assist in monitoring network activity and improving overall security visibility within datacenter environments.

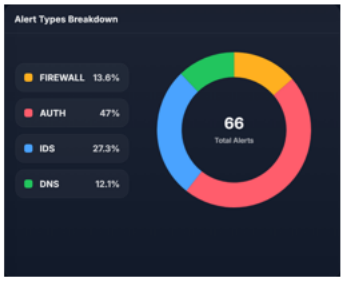


Figure 4: Alerts pie chart

Conclusion and personal reflection

This project examined security challenges in datacenter networks and developed a Python-based log analysis tool and monitoring dashboard to analyse network log data and detect suspicious activity. The system successfully processed simulated logs and applied detection rules to identify patterns such as repeated firewall denies and abnormal authentication attempts.

Through this project I developed skills in Python programming, cybersecurity research, log analysis, and data visualisation. The work also strengthened problem-solving and independent research abilities. If the project were undertaken again, future improvements could include integrating real time log data and expanding the detection capabilities of the system.

Acknowledgements

I would like to thank Dr. Kevin Murphy for his guidance, support and advice throughout the development of this project.

UNIVERSITY OF LIMERICK OLLSCOIL LUIMNIGH